

Homework 2 Report

GitHub Repo

<https://github.com/halliepailes/CSCE-465-HW2>

Task 0 - Lab Preparation

For Task 0, I successfully set up my virtual environment with python3 and the cryptography and pytest libraries. I also successfully created the standard ffdhe3072 parameter file for the Diffie-Hellman setup I will use in Task 2.

Results

```
GROUP: ffdhe3072
(.venv) student@student-VirtualBox:~/csce465-agentsec/hw2$ python3 --version
Python 3.12.3
(.venv) student@student-VirtualBox:~/csce465-agentsec/hw2$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
(.venv) student@student-VirtualBox:~/csce465-agentsec/hw2$ pip show cryptography pytest
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to Python developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors <cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/student/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
---
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laughner, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/student/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
(.venv) student@student-VirtualBox:~/csce465-agentsec/hw2$
```

Task 1

For Task 1, I created a python file called baseline_ctr.py to demonstrate that encryption alone is not sufficient for protocol security. The script encrypts a fixed-length command using AES-CTR mode with an authenticating MAC tag. To prove the vulnerability, I implemented a relay attack that manipulates specific ciphertext bits to alter the command without knowing the secret key. I also successfully replayed the same ciphertext to force the receiver to process the exact same command twice, proving that just confidentiality does not protect against integrity or replay attacks.

Results

```
(.venv) student@student-VirtualBox:~/csce465-agentsec/hw2$ python3 baseline_ctr.py
Original Plaintext: b'{"action":"READ","path":"notes.txt"}'
Original Ciphertext: b'\xe5&\xbd\xc8\xed\xb0\xb1[\xc5\xe2\x9f\xddX4\x12\xb2\xfb&\xe9\xe4\x9c\xab\n\xd25\xc4@\xc0g\xaf\xe7R\x84\xc7\xf6'

XOR Relation
Index 11: 82 XOR 69 = 23
Index 12: 69 XOR 68 = 1
Index 13: 65 XOR 73 = 8
Index 14: 68 XOR 84 = 16

First Run with New Ciphertext - Processed Payload: b'{"action":"EDIT","path":"notes.txt"}'
Second Run with Same Ciphertext - Processed Payload: b'{"action":"EDIT","path":"notes.txt"}'
(.venv) student@student-VirtualBox:~/csce465-agentsec/hw2$
```

Analysis

Design

In my python script, I have created a simulated pipeline using the python cryptography library. The design uses AES operating in Counter Mode (CTR) with a secret key. The sender transmits the fixed-length command as a byte string: `b'{"action":"READ","path":"notes.txt"}'`. The code generates the secret key and the nonce required for the cipher, then encrypts the plaintext. It loops through the “READ” action that we want to replace, and changes the corresponding bytes in the ciphertext to instead say “EDIT”.

Experiment Results

Within this experiment, the code targeted the 11th - 14th index of our payload, where the “READ” is located, and it XORed each character at those indexes with each character within my new action string “EDIT”. After XORing the characters, it then XORed it with the character at that index within the original ciphertext and saved it as our new ciphertext. Within the first run where it performs the relay, it takes in the modified ciphertext, and successfully decrypts it to the altered command: `b'{"action":"EDIT","path":"notes.txt"}'`. Within the second run, where it performs the replay, it took in the same modified ciphertext and decrypted it without detecting that the message had been processed twice.

Root Cause

The root cause of this is the mathematical design of the protocol. Due to the AES-CTR encryption we did, the ciphertext is the plaintext XORed with the keystream. Thus, by looping through “READ” and XORing it with “EDIT”, then XORing that with the original ciphertext, it essentially did a collection of XORs with the overall outcome being: $\text{plaintext}[\text{index}] \oplus \text{new_action}[i] \oplus \text{plaintext}[\text{index}] \oplus \text{keystream}$. Since XORing the plaintext with itself is 0, we are then left with the result being $\text{new_action}[i] \oplus \text{keystream}$, allowing us to XOR our new action with the proper keystream, without having to know what it is. This is what allowed us to successfully complete our relay attack. For the replay attack, AES-CTR relies on unique nonces for confidentiality but does not track any other internal state data, thus it doesn't track any session IDs or timestamps. Because of this, a valid ciphertext can be reused

indefinitely, forcing the receiver to execute the same command repeatedly, which is what allowed us to successfully complete the replay attack.

Task 2

For this task, I created handshake.py, where I designed and implemented a local, authenticated key exchange utilizing the cryptography library in python. The code uses authentication and ephemeral key agreement to create a secure session between a simulated gateway and node.

Results

```
Second run with same ciphertext - Processed Payload: b' action : EBIT', path : no  
(.venv) student@student-VirtualBox:~/csce465-agentsec/hw2$ python3 handshake.py  
Handshake complete - SessionID: 0ae1703e1859d692
```

Analysis

Diffie-Hellman Key Establishment

The core key agreement between the gateway and node relies on a Diffie-Hellman setup. The code parses the 3072 bit parameter file, ffdhe3072.pem, and uses it to generate the public and private ephemeral keys for both the gateway and node. Those public keys are then converted from the raw integers into 384 byte big-endian byte strings. These keys are what are used by each party to compute the shared secret integer, Z . Because of the math used within the Diffie-Hellman protocol, both sides are able to calculate the exact same secret number, but an eavesdropper listening to the network can't compute it.

Digital Signatures

While Diffie-Hellman establishes a shared secret between our gateway and node, it is still vulnerable to impersonation since it doesn't authenticate who generated the keys. To fix this, the code creates a 3072 bit RSA key pair for both the gateway and node. These keys function as identification for the parties to ensure that they are valid. It implements the RSA-PSS digital signature scheme, which adds salt before signing to always ensure that the same message doesn't sign the same when used multiple times. It uses this along with the SHA-256 hash, which converts the message into a fixed-size 256 bit code.

Entity Authentication

Entity authentication is the process of proving that a party that is attempting to communicate is a verified entity. In the code, this is achieved through the RSA public and private key pairs. By having the node and gateway verify each other's signatures, both parties are able to verify that their identities are correct before any data is passed between them.

Nonces

A nonce is a "number used once", and in my protocol design, both the gateway and the node generate their own 16 byte nonce at the start of every connection attempt. These nonces ensure that every part of the handshake is mathematically unique, which is what allows the transcript to look different even if both the gateway and node say the exact same things.

Replay Resistance

Replay resistance is what prevents an attacker from capturing a valid encrypted message and sending it again later to trick the server into repeating the action over and over. Because the code's unique 16-byte nonces are used directly into the signed transcript hash, TH, an attacker can't take a successful handshake message and send it again later. Thus, if an old signature is sent to the receiver again, it will detect the mismatch between the nonces from the current session and the nonces from the old session and the signature verification will fail.

Forward Secrecy

Forward secrecy is a property that ensures that even if an attacker steals private keys, they still can't decrypt any network traffic that they collected in the past. The code achieves this because the actual session keys are derived from the ephemeral, or temporary, Diffie-Hellman keys, which exist only in the current session memory and get deleted after the handshake is completed. Since the RSA keys are only used to authenticate the exchange and not encrypt the data, if an attacker steals the RSA keys, they still have no way to calculate the secrets from the Diffie-Hellman protocol.

Key Separation

Key separation ensures that distinct keys are used for distinct tasks, which prevents a vulnerability from one task from interfering with the security of another. The code achieves this by passing the masker key, K_master, into an HMAC-SHA-256 function paired with strings labeling their purpose, like "gateway-to-node encryption" and "gateway-to-node mac." This isolates each channel so that if there is a compromise within the key encryption layer, it won't affect the integrity of the MAC channel.

Final Analysis

By signing both identities, roles, nonces, and the Diffie-Hellman values within the transcript, the code creates a complete, unchangeable record of who each entity is and how they will communicate. If a man-in-the-middle attacker tries to intercept the Diffie-Hellman keys and replace it with their own, the transcript will be altered and the TH value will be shifted, which causes the signature check to immediately fail. Also, including the strings "gateway" and "node" explicitly within the messages uniquely separates each message by their role. Thus, if an attacker captures the gateway signature and sends it back to trick the gateway, it will be rejected since the gateway is expecting the signature with the node role, which allows us to prevent reflection attacks. If the digital signatures were removed, the protocol is then left with just the Diffie-Hellman setup. Both sides would still be able to calculate the

secret key, however there would be no way to verify who they are talking to, allowing a man-in-the-middle attacker to get in and read and modify the network traffic without either side knowing.

Task 3

For Task 3, I implemented a secure network record layer within a `secure_record.py` python file by creating the `seal` and `open_record` functions using an Encrypt-then-MAC design. The `seal` function packages the application data into a byte sequence that contains a header, an initialization vector, the ciphertext, and an authenticating HMAC-SHA-256 integrity tag. The `open_record` function protects the system against exploits over transit by checking that the application data byte sequence wasn't tampered with and by verifying the direction and sequence counter. This helps to eliminate message replays. It also uses the verification function from the cryptography library to validate the message before any data is decrypted. If a modified header or ciphertext, incorrect direction, or out of order sequence number is detected, then an exception is immediately raised, ensuring that the unauthenticated plaintext is never released.

Task 4

For Task 4, I wrote a set of 6 automated adversarial tests inside `test_security.py` to physically verify that the record layer protects the system against network threats. Using python assert statements and a series of try/except blocks, the script runs six attack scenarios, including verifying valid bidirectional messages, ciphertext and header manipulation, message replays, reflection attacks, and handshake failures. Each test attempts to breach the protocol and successfully verifies that the underlying code triggers an immediate and safe failure. All tests pass on execution, mathematically proving that our handshake and record architecture protects against transit exploits.

Results

```
(.venv) student@student-VirtualBox:~/csce465-agentsec/hw2$ python3 tests.py
Starting Tests...

Test 1 - Valid Handshake & Bidirectional Messages Passed: Valid bidirectional messages worked succesfully
Test 2 - Modified Ciphertext Passed: Successfully detected & blocked modified ciphertext
Test 3 - Modified Authentication Header Passed: Successfully detected & blocked modified header
Test 4 - Replayed Record Passed: Successfully detected & blocked replayed record
Test 5 - Replayed Record Passed: Successfully detected & blocked redlected record
Test 6 - Handshake Failures Passed: Successfully caught invalid handshake

All tests passed!
```

Security Note

Attacker Model & Non-Goals

Within this attacker model, the attacker is assumed to have full control over the network, giving them the ability to intercept, read, modify, and inject data records during the communication process. The security goals are to establish confidentiality, message entity authentication, verification, and replay resistance for records passed between the gateway and node. The non-goals include providing traffic eavesdropping resistance, mitigating network availability concerns, and protecting against physical hardware tampering or malware compromise.

Why CTR Mode Requires Unique IVs

AES-256 in Counter Mode functions by transforming a block cipher into a stream cipher. It encrypts a sequence composed of an initialization vector and a sequence counter in order to generate a keystream. This keystream is then XORed with the plaintext to produce the ciphertext. If an initialization vector is ever reused under the same secret key, then an identical keystream is generated for both messages. Because of this, an eavesdropper can capture both ciphertexts and XOR them together, cancelling out the keystreams and returning the plaintexts XORed together. With this information, an attacker could potentially recover the plaintexts using frequency analysis. To prevent this, we need to ensure each initialization vector is unique, which we do by appending an incrementing sequence counter to the session ID. This guarantees that an initialization vector is never repeated within a session.

Why Verify-Before-Decrypt Matters

When we stick to only the Encrypt-then-MAC configuration, it requires the receiver to do HMAC validation before attempting to decrypt the payload. This is because if decryption is attempted first, then malicious modifications that were injected by an attacker can be parsed by our AES. So, by verifying the MAC tag on the encrypted message before decrypting, we are ensuring that any unauthenticated payload data isn't released or processed during decryption.

Why Independent Encryption & MAC Keys are Used

Using the same key for both symmetric encryption and MAC tagging introduces mathematical dependencies which is a big security risk. When you reuse a key, the math behind the encryption can accidentally leak clues about how the security tags are generated. To enforce strict key separation, we run a key derivation function that mixes the master key with unique labels to create separate, independent keys. This keeps the keys isolated from each other, so even if an attacker is able to obtain the encryption key, they still won't be able to figure out the validation key.

Comparison with Authenticated Encryption (AEAD) Modes

While the Encrypt-then-MAC layer that we built allows us to have security when executed correctly, there are other encryption modes that are used more frequently in modern examples, like modes from Authenticated Encryption with Associated Data, or AEAD. Combining separate encryption and

MAC layers leaves room for flaws within the programming, like incorrect length calculations or wrong parsing orders. Furthermore, it complicates key management because you are forced to safely handle and store two completely separate keys for every single connection. AEAD modes resolve this by combining encryption and integrity protection within a single mathematical loop under just one master key. For example, AES-GCM is an AEAD mode which takes plaintext along with other, unencrypted data like headers to produce the ciphertext and authentication tag at the same time. This provides built-in header protection by automatically authenticating the headers as associated data without requiring any manual checking on our end. This helps to eliminate risks during implementation.

Dangerous Authorized Tool Calls

Within cryptography, we can only protect the delivery of a message. Due to this, we can't see if the message itself is safe. Thus, if a trusted user or attacker who successfully stole their keys sends a dangerous command, since it passes the security checks implemented, the system will run it with no issue. So even if a record is completely valid, it doesn't necessarily mean that it is safe, as the contents may contain dangerous tool calls.